Take-Home Assignment: Mini Project Management App

Role: Lead Engineer — Full Stack
Time budget: 3–4 days (AI tools allowed and expected)
Format: Solo project, public or private Git repo

Overview

Build **Flowboard** — a mini project-management app for a single team ("workspace").

This is a **full-stack** assignment. You must ship **both**:

Layer	Allowed stack (pick one per layer)
Backend API	**Node.js** (TypeScript preferred) **or Go**
Frontend	**Vue or React** only

A user should be able to:

1. Organize work in a **hierarchy**
2. Manage **tasks** inside lists (including **custom fields**)
3. Configure **status options** that can vary by space / folder / list
4. View tasks on a **kanban board** and in a **list view**
5. Control **who sees what** with a simple permission model

Think ClickUp / Asana / Linear at hobby scale — but design it like something that could grow into a production system.

> **Why the extra product surface?** With AI tools, CRUD + a board is too easy to generate. Custom fields and scoped statuses force real modeling, validation, inheritance, and **testable edge cases**. We grade those as heavily as the happy path.

Engineering expectations (required)

We evaluate lead-level judgment as much as feature completeness. Your solution must demonstrate the following.

1. System design diagram

Include a clear architecture diagram (Mermaid, draw.io export, or ASCII) covering:

- Clients (web UI; note other channels if designed)
- API gateway / BFF (if any)
- Application services
- Datastores
- Auth boundary
- Async / IPC paths (if used)

Call out **trust boundaries**, request flow for "load board + drag card", and where permissions are enforced. Also show where **status resolution** and **custom-field schemas** live relative to the hierarchy.

2. API standardisation

Pick one style and apply it consistently (REST preferred; GraphQL allowed if justified):

- Resource-oriented URLs and predictable HTTP verbs (or equivalent GraphQL conventions)
- Consistent JSON error shape: `{ error: { code, message, details? } }`
- Pagination on list endpoints (cursor preferred; offset acceptable)
- Versioning strategy (`/v1/...` or documented equivalent)
- OpenAPI/Swagger **or** an API overview table in the README that matches the running code

3. Auth

Implement a real auth boundary (mock identity is fine; unauthenticated access is not):

- Client sends identity via **`X-User-Id: alice|bob|carol`** and/or a short-lived **JWT** `{ userId, role }`
- Backend resolves the caller **server-side** — never trust role/permissions from the request body alone
- Unauthorized → `401` ; forbidden → `403`
- Document how you would swap mock auth for OIDC / session auth in production

4. Multi-channel support

Design APIs so the same domain can serve more than the SPA:

- **Required:** Web UI (Vue or React) as primary channel
- **Required (design + light implementation):** at least one additional channel surface, e.g.:
  - thin **CLI** or **curl cookbook** hitting the same APIs, **or**
  - a second client stub (e.g. simple mobile-oriented JSON endpoints / BFF notes), **or**

- webhook / event payload examples for “task updated”

Document channel boundaries: shared contracts, auth, and what stays UI-only.

5. Scalability & service boundaries

The MVP may be a modular monolith, but you must design for scale:

- Clear module/service boundaries (e.g. hierarchy, tasks, status config, custom fields, permissions, identity)
- Stateless API processes; horizontal scale story for the API tier
- DB indexing / query notes for tree + board loads + custom-field filters
- What you’d split into separate services in week 2–4 and **why**

6. IPC / microservice communication (if needed)

If you introduce more than one backend process (or design for it):

- Choose and document an IPC approach (HTTP/gRPC, message queue, pub/sub, etc.)
- Show at least one **async** path in design (e.g. “task status changed” → activity / search indexer stub)
- Cover failure modes: retries, idempotency, eventually consistent reads

A single well-structured service with a documented future IPC plan is acceptable if time is tight — say so explicitly.

7. Dockerized end-to-end run (required)

The entire project must run via Docker. Reviewers should be able to start the stack with one command, typically:

```
docker compose up --build
```

Expected compose services (adjust names as needed):

- Frontend
- Backend API
- Database (and cache/queue if you use them)
- Any worker / sidecar you introduce

Include `.env.example` (no secrets). Seed data should apply on first boot or via a documented `make seed` / compose init job.

Scope & time guidance

Day	Focus
Day 1	Docker Compose, hierarchy + status inheritance model, custom-field schema, seed, auth
Day 2	Task APIs (incl. custom values + move/status validation), kanban + list UI + task detail
Day 3	Permissions, config UIs (statuses / fields), tests for edge cases, docs / system design
Day 4 (optional buffer)	Polish inheritance edge cases, filters, demo video

AI assistants (Cursor, Copilot, ChatGPT, etc.) are **encouraged**. We evaluate **your judgment**, domain modeling, and tests — not typing speed.

Must-have requirements (MVP)

1. Hierarchy (containers)

Model a tree scoped to one workspace:

```
Workspace
└── Space (e.g. "Engineering")
    └── Folder (e.g. "Q2 Launch")
        └── List (e.g. "Backlog")
            └── Tasks
```

Rules:

- Each node has: `id`, `name`, `type`, `parentId`, `position` (for sibling ordering)
- Valid parent types: `workspace → space → folder → list` (lists hold tasks, not other containers)
- CRUD for each container type
- **Reorder** siblings (drag-and-drop or explicit move API)
- Soft-delete or archive (your choice — document it)

2. Tasks

Each task belongs to exactly **one primary list** and has:

Field	Required	Notes
`title`	Yes	Max 500 chars

Field	Required	Notes
`description`	No	Plain text or markdown
`statusId`	Yes	Must be valid for the **resolved status set** of the task's list
`priority`	No	`urgent` | `high` | `normal` | `low` | `none`
`assigneeIds`	No	Array of user IDs (mock users are fine)
`dueDate`	No	ISO datetime
`customFields`	No	Map of `fieldKey → value` (see §4)
`position`	Yes	Order within status column / list
`createdAt` / `updatedAt`	Yes	

Task operations:

- Create, read, update, delete
- Move between lists (updates `primaryListId`) — **must re-validate** `statusId` + custom fields against the destination list's resolved config
- Change status (updates kanban column)
- Reorder within a column
- Set / clear custom field values with type validation

Stretch within MVP if time allows: one level of subtasks (`parentTaskId`), max depth 1.

3. Scoped status options (required)

Statuses are **not** a single global enum. Each **space, folder, or list** may define (or override) a status set.

Status entity: `id`, `key`, `name`, `category` (`not_started` | `active` | `done` | `closed`), `color`, `position`, `ownerType`, `ownerId`.

Minimum categories present in every resolved set:

- at least one `not_started` (e.g. Todo)
- at least one `active` (e.g. In Progress)
- at least one `done` (e.g. Done)

Inheritance / resolution rules (document your exact algorithm):

1. Resolve statuses for a **list** by walking ancestors: `list → folder → space → workspace default`
2. A child may **inherit** parent statuses, **override** labels/colors/order, **add** statuses, or **hide/disable** inherited ones (pick a clear model — e.g. "override set replaces parent" vs "merge by `key`")

3. Tasks store a `statusId` (or stable `statusKey`) that must exist in the **resolved** set for their current list
4. Kanban columns = **resolved** statuses for the selected list (not raw parent configs)

Operations:

- CRUD status definitions on space / folder / list (with permission checks)
- Reorder statuses
- Preview / `GET .../resolved-statuses` for a list (used by board + validators)

Hard cases you must handle (and test):

- Moving a task to a list whose resolved set does **not** include the old status → reject **or** map to a default (document choice; both need tests)
- Editing/removing a status that is still referenced by tasks → reject, archive+remap, or soft-disable (document + test)
- Two sibling lists under the same folder with **different** status sets both render correctly on the board

4. Custom fields (required)

Support **custom fields** defined on the hierarchy and applied to tasks.

Field definition (owned by space, folder, or list — same scoping idea as statuses):

Attribute	Notes
`id` / `key`	Stable `key` used on task payloads
`name`	Display label
`type`	Support at least: `text`, `number`, `single_select`, `multi_select`, `date`, `checkbox`
`required`	boolean
`options`	For select types: `{ id, label, color? }[]`
`ownerType` / `ownerId`	space | folder | list
`position`	Ordering in forms / list columns

Resolution:

- Resolved field schema for a list = inherit from ancestors + local definitions (document merge/override rules; `key` collisions must be explicit)
- Task create/update validates values against the **resolved** schema for that list
- Required fields enforced on create (and on update if you mark the field required)

UI (minimum):

- Task detail: render inputs for resolved custom fields and save values
- List view: show at least **two** custom-field columns (configurable or seeded)
- Kanban card: optional — show up to one custom field badge if useful

Hard cases you must handle (and test):

- Wrong type (e.g. string for `number`) → `400` with field path in `details`
- Select value not in options → `400`
- Moving task to a list where a required field is missing → reject or require values in the move payload
- Removing / changing a field definition that existing tasks use → defined migration behavior (strip, keep orphan values, or block delete)

5. Views (frontend — Vue or React)

Implement **two views** for the selected list:

A. Kanban board

- Columns = **resolved** statuses for the list
- Cards = tasks (title, assignee avatars/initials, priority badge, due date; optional custom-field chip)
- Drag card → another column updates status
- Drag within column updates position

B. List view

- Table or dense list: title, status, assignees, priority, due date, **custom fields**
- Sort by due date and priority (bonus: sort/filter by one custom field)
- Click row → task detail drawer/modal (built-ins + custom fields)

C. Sidebar

- Collapsible tree of workspace → spaces → folders → lists
- Selecting a list loads its board/list view

D. Light config UI (required)

Enough UI to prove the model (can be simple forms, not a polished admin console):

- Manage statuses for at least one non-list level **and** a list (or clear “edit resolved vs local” UX)
- Manage custom field definitions for at least one container level

6. Permissions (required — keep it simple)

Support **three mock users** (seed in DB):

User	Role	Expected behavior
Alice	`admin`	Sees and edits **everything** in the workspace
Bob	`member`	Sees lists/spaces they're granted access to; can edit tasks in those lists
Carol	`member`	Same as Bob, different grants

Permission model (minimum):

- Grants attach to a **container** (space, folder, or list): `{ resourceId, userId, mode: allow | deny }`
- **Public** containers: visible to all workspace members unless explicitly denied
- **Private** containers: visible only with an explicit `allow` grant (admins bypass this)
- Tree API returns only nodes the current user can see
- Attempting to access a denied resource returns `403`
- Status / custom-field **config** mutations: admin-only **or** members with edit access on that container (document choice)

You do **not** need team-based grants or complex inheritance — but **document** how you would extend your model.

7. Persistence

- Any DB: PostgreSQL, MongoDB, SQLite, etc. (must run as a Docker service for the default path)
- Include a **seed script** that creates at least:
  - 1 workspace, 2 spaces, 2 folders, **3 lists**
  - **Workspace (or space) default statuses** plus **at least one list that overrides/extends** statuses
  - **≥3 custom field definitions** across ≥2 hierarchy levels (include at least one `single_select` and one `required` field)
  - **20+ tasks** with varied statuses, priorities, and custom field values
  - 3 users + sample grants (Bob and Carol must differ)
- Migrations or schema definition checked into repo

8. Tests (expanded — required)

AI can generate the happy path. Your tests should prove you thought about the domain.

Backend (minimum — aim for these named scenarios):

Area	Required scenarios
Permissions	Tree hides private nodes for Bob; Alice sees all; denied list → `403` on task read
Status resolution	List A and List B under same folder resolve **different** column sets
Status validation	Create/update task with unknown `statusId` → `400`
Cross-list move	Move task into list without old status → per your policy (assert status outcome)
Custom fields	Reject invalid type / invalid select option; accept valid payload
Required fields	Create without required custom field → `400`
Config safety	Delete/disable in-use status → per your policy (assert no orphan board state)

Frontend (minimum):

- One component test **or** one E2E path that covers: select list → see expected kanban columns → open task → edit a custom field → save
- Bonus: E2E that switches user header Alice → Bob and asserts a private list disappears

Document how to run tests in Docker or via `make test`.

9. Documentation (README)

Your README is part of the submission. Include:

1. **One-command run:** `docker compose up --build` (document ports and default users)
2. **System design diagram** (see Engineering expectations)
3. Data model explanation — especially **status inheritance** and **custom-field resolution**
4. API overview / OpenAPI link + standardisation notes
5. Auth model + multi-channel notes
6. Scalability / IPC plan (what you built vs what you'd add next)
7. **Trade-offs:** what you cut, what you'd do on day 5 / week 2
8. **AI usage log:** tools used and where they helped vs where you corrected them (call out any AI-generated tests you had to fix)

Nice-to-have (stretch goals)

Pick **at most two** if MVP is solid. Tell us which you attempted in the README.

- Filter list/board by custom field values
- Full-text search on task title/description (+ optional custom text fields)
- Optimistic UI + conflict handling on drag-drop
- Activity feed ("Alice moved task X to Done") via async IPC

- Bulk update (multi-select → change status/assignee/custom field)
- Real-time updates (WebSocket/SSE)
- OpenAPI / Swagger with generated client
- Deploy compose stack to Fly.io / Railway / Render

Technical constraints

Topic	Guidance
Backend	**Node.js or Go only**
Frontend	**Vue or React only** (no Svelte/Angular/etc. for this assignment)
Delivery	Full stack — API **and** working UI required
Runtime	Entire stack runnable via **Docker Compose**
Styling	Any UI library or custom CSS — clarity over pixel perfection
Scope	Single workspace, single tenant — no billing, no email, no file uploads
Users	Seeded mock users — no signup/login product flow required
Mobile	Desktop-first; responsive is a plus, not required

Deliverables

Submit a link (GitHub/GitLab) containing:

1. Source code (frontend + backend)
2. `docker-compose.yml` (or equivalent) and `.env.example`
3. README (see section 9)
4. `AI_USAGE.md` (optional but appreciated) — prompts, corrections, what you rejected from AI output
5. Short **demo video** (4–6 min, Loom/YouTube unlisted) walking through:
   - `docker compose up`
   - tree navigation
   - kanban columns that differ across two lists (status scoping)
   - edit a custom field on a task
   - permission difference between Alice vs Bob

Do not submit: credentials or API keys (use `.env.example`).

FAQ

Can I use a boilerplate?
Yes, but strip what you don't need and explain your choices.

Can I skip the frontend and do API-only?
No. This assignment is **full stack** — Vue/React UI + Node/Go API are both required.

Can I use Python / Java / Svelte / Angular?
Not for this assignment. Backend must be Node.js or Go; frontend must be Vue or React.

Must everything really run in Docker?
Yes. Reviewers will start from a clean machine with Docker only. Local `npm run` / `go run` paths may exist for DX, but Compose is the supported path.

Do I need real microservices?
Not required for MVP. You **do** need clear boundaries and a credible scale/IPC story. Implementing a second process or async worker is a plus if MVP is solid.

Can I only put statuses on lists?
No — MVP requires **scoped** statuses with inheritance/override from a higher level (space and/or folder) down to list. Lists-only configs will be scored as incomplete.

Are custom fields optional?
No. Schema + task values + validation tests are required. Fancy field builders can stay minimal.

What if I run out of time?
Prefer a correct inheritance/validation model with tests over a half-finished search feature. Tell us what you'd do on day 5.

What stack do you use internally?
We'll discuss our production stack in the interview; use Node/Go + Vue/React as specified so we can compare submissions fairly.

Questions? Contact your recruiting coordinator.